

SYSTEM DESIGN (HLD) — COMPLETE NOTES

Source: "Concept & Coding" (Shrayansh Jain) — LLD & HLD Playlist

Dense, interview-ready reference notes. One section per video, numbered to match the playlist (0001–0042). English-dubbed duplicates are merged into a single topic and labelled with both video numbers.

TABLE OF CONTENTS

Foundations

- 0001 — LLD & HLD Roadmap
- 0002 / 0003 — Network Protocols
- 0004 / 0005 — CAP Theorem
- 0006 / 0007 — Monolith vs Microservices + Microservices Patterns
- 0008 / 0009 — Scale from 0 to 1 Million Users
- 0010 / 0011 — Consistent Hashing

Core Design Questions

- 0012 — Design a URL Shortener
- 0013 — Back-of-the-Envelope Estimation
- 0014 — Design a Key-Value Store (DynamoDB)
- 0015 — SQL vs NoSQL (Which DB to Use)
- 0016 — Design a Chat Application (WhatsApp)

Distributed Systems Building Blocks

- 0017 — Design a Rate Limiter
- 0018 — Idempotency Handling
- 0019 — High Availability Architecture
- 0020 — Distributed Messaging Queue (Kafka / RabbitMQ)
- 0021 — Proxy Servers
- 0022 — Load Balancers
- 0023 — Caching
- 0024 — Distributed Transactions
- 0025 — Database Indexing

Concurrency & Security

- 0026 — Concurrency Control (Optimistic vs Pessimistic)
- 0027 — Two-Phase Locking (2PL)
- 0028 — OAuth 2.0
- 0029 — Cryptography
- 0030 — JWT (JSON Web Token)
- 0031 — Ticket-Booking Concurrency Failure (Case Study)

Microservices Infrastructure & Resilience

- 0032 — API Gateway
 - 0033 — Service Mesh
 - 0034 — DNS (Domain Name System)
 - 0035 — How Many Microservices? (Sizing)
 - 0036 — Web Attacks: CSRF, XSS, CORS, SQL Injection
 - 0037 — Dual-Write Problem (Event-Driven Microservices)
 - 0038 — Service Discovery (Eureka)
 - 0039 — Rate Limiter as Resilience (Resilience4j)
 - 0040 — Bulkhead Pattern
 - 0041 — Retry Pattern
 - 0042 — Circuit Breaker Pattern
-

FOUNDATIONS

0001 — LLD & HLD ROADMAP

The intro video that frames the whole playlist. Two separate tracks: Low-Level Design (LLD) and High-Level Design (HLD).

LLD track — how to progress

1. Prerequisite: OOP fundamentals — inheritance, polymorphism, abstraction, encapsulation. Any OOP language works (C++, Java, Python).
2. SOLID principles — the base for all good LLD. Always step #1 in the LLD playlist.
3. Design patterns (23 GoF patterns) — the video's philosophy is *not* to teach all 23 up front. Instead: learn a few important patterns, then solve interview questions on top of them.
 - Questions solvable *directly* via a pattern: Notify-Me (Observer), Pizza billing (Decorator), Vending machine (State), ATM, File system.
 - Questions that *use* patterns as building blocks: Parking lot, Splitwise, BookMyShow, Snake & Ladder,

Elevator system, Car rental, Chess, Tic-Tac-Toe, Logging system.

4. Key point: LLD has no single "correct" answer and no fixed syllabus — it's learned through structured practice on real interview questions.

HLD track — why it's different

HLD *requires learning concepts before you can attempt questions*. E.g., you can't design DynamoDB without knowing consistent hashing first. So the big design questions sit at the *bottom* of the roadmap, after the building blocks.

HLD building blocks to master (roughly in order):

- Network protocols (TCP, HTTP, WebSocket, WebRTC), client-server vs P2P
- CAP theorem
- Microservices patterns (Saga & Strangler are must-knows)
- Scale from 0 → 1M users
- Consistent hashing
- Back-of-the-envelope estimation
- SQL vs NoSQL
- Messaging queues (Kafka), proxies, CDN, storage types (block/file/object, S3, RAID), Bloom filters, Merkle trees, gossip protocol, caching
- DB scaling: horizontal/vertical partitioning, replication/mirroring, leader election, indexing

Design questions built on these: URL shortener, Key-Value store, WhatsApp, Rate limiter, Autocomplete/Typeahead, etc.

Takeaway: LLD = OOP → SOLID → patterns → questions. HLD = concepts/components first → then design questions. Master the building blocks; the design questions are just combinations of them.

0002 / 0003 — NETWORK PROTOCOLS

Why it matters: The very first HLD decision is often *which protocol* to use. Designing WhatsApp? → WebSocket. Designing Google Meet / video streaming? → WebRTC/UDP. You must justify the choice.

Definition: A network protocol defines the *rules* that let two systems communicate over a network — even if both "speak the same language," they need agreed rules for how to send/receive.

The OSI model has 7 layers (Application, Presentation, Session, Transport, Network, Data-Link, Physical). For HLD interviews, two layers matter: Application and Transport.

Application Layer

Split into two families:

A) Client–Server protocols — client *initiates*, server *responds*.

Protocol	Key facts
HTTP	Hypertext Transfer Protocol. Connection-oriented. Serves web pages; hyperlinks jump page→page. Request/response, client-initiated. Use HTTPS (secure) in practice.
FTP	File Transfer Protocol. Maintains two connections: a control connection (stays open) + a data connection (opens/closes per transfer). Data is not encrypted → insecure → rarely used today.
SMTP	Sending email only. Works with IMAP/POP3 for receiving. Flow: User Agent → MTA (Message Transfer Agent) → MTA → User Agent → mailbox. IMAP reads from server (multi-device); POP3 downloads then deletes from server (legacy).
WebSocket	Bidirectional / full-duplex — client can talk to server <i>and</i> server can push to client. Still client-server, NOT peer-to-peer (clients never talk to each other directly; the server relays).

Common trap: WebSocket is *not* P2P. Clients only talk *through* the server.

B) Peer-to-Peer protocols — every node can be client *and* server; nodes talk *directly*.

Protocol	Key facts
WebRTC	Real-time comms. Peers communicate <i>directly</i> (no server round-trip for media) → low latency. Runs over UDP underneath. Use for video/audio calling & live streaming.

Most important for HLD: HTTP, WebSocket, WebRTC.

Transport Layer — TCP vs UDP

	TCP/IP	UDP/IP
Connection	Virtual connection established first (handshake)	Connectionless
Data unit	Packets, sequenced	Datagrams, sent in parallel over multiple paths
Ordering	Guaranteed (reassembled in order)	Not guaranteed (packets may arrive out of order)
Reliability	ACK per packet + retransmit on loss	No ACK, no retransmit — best-effort
Speed	Slower (overhead of ordering/ACK/connection)	Faster (no overhead)
Use when	Data integrity matters (web, files, DB, messaging)	Speed > completeness; occasional loss OK
Use cases	HTTP, FTP, email, most apps	Live video/audio calls, streaming, gaming; WebRTC

Why UDP for live streaming/video calls: if a packet is dropped mid-call, you don't want to re-fetch old audio — you move on. Loss is acceptable; latency is not.

Decision cheat sheet

- Messaging / chat app (server must push to client) → WebSocket
- Live streaming / video call → WebRTC (UDP under the hood)
- Web pages / normal request-response → HTTP(S)
- Need reliable, ordered delivery → TCP; need speed, tolerate loss → UDP

0004 / 0005 — CAP THEOREM

Why it matters: In an HLD interview you may be asked to *define* CAP outright, but more importantly, the CAP trade-off must be decided early — it shapes the entire design and is very hard to change after the system is built.

Definition: CAP describes three *desirable properties* of a distributed system with replicated data. You can only guarantee two of the three simultaneously.

Setup: an application queries DB nodes B and C (say, B in India, C in the US) that replicate the same data to each other.

Property	Meaning
C — Consistency	After a successful write to any node, a read from <i>any</i> node returns the latest value. (Write A=5 to B → B replicates to C → reading from C also returns 5.)
A — Availability	Every request receives a response (success <i>or</i> failure) — all nodes stay operational/responsive. The system is never "down."
P — Partition Tolerance	The system keeps working even when the communication between nodes breaks (a network partition). The user still queries successfully; internally, nodes may be out of sync.

Why you can't have all three

When a partition happens (B and C can't sync), and a write comes in:

- To keep A + P: serve the request anyway → C returns stale data → Consistency lost ⇒ you get AP.
- To keep C + P: you must take a node down (route everything to one node) so no stale reads happen → Availability lost ⇒ you get CP.
- To keep C + A: you must stop serving during the partition (reject writes/reads) → the system isn't tolerant of the partition ⇒ CA (only works if a partition never happens — i.e., effectively a single node / non-distributed).

The three combinations

Combo	You sacrifice	When to use	Examples
CP	Availability	Correctness is critical	Banking, inventory, booking, ZooKeeper, HBase, MongoDB (default)
AP	Consistency	Uptime > perfect data; eventual consistency OK	Social feeds, likes, DNS, Cassandra, DynamoDB
CA	Partition tolerance	Single-node / no network partition possible	Traditional single-node RDBMS

Interview takeaway

In any real distributed system, network partitions are inevitable, so P is non-negotiable. The real question is always: *"Do you trade off C or A?"* → choose CP or AP based on the product's needs. Never trade away P.

0004 / 0005 — CAP THEOREM

0006 / 0007 — MONOLITH VS MICROSERVICES + MICROSERVICES DESIGN PATTERNS

Monolithic architecture

A single deployable unit containing all functionality — order generation, product/inventory management, login, billing, payments — in one codebase and (usually) one database.

Pros: simple to develop/test/deploy early; no inter-service network calls (low latency); easy end-to-end transactions (single DB); simple monitoring.

Cons:

- Hard to scale selectively — you must scale the whole app even if only one module is hot.
- Overloaded builds/IDE, slow CI/CD as the codebase grows.
- Tight coupling — a change in one area risks breaking others.
- Single point of failure — one bug can take down everything.
- One tech stack locked in for the whole app.
- Risky deployments; hard for large teams to work in parallel.

Microservices architecture

Decompose the app into independently deployable services, each owning a business capability and its own database, communicating over the network. Services should be loosely coupled.

Pros: independent scaling & deployment; tech heterogeneity (each service picks its own stack); fault isolation (one service down ≠ whole system down); team autonomy; cost-efficient scaling (scale only hot services).

Cons:

- Distributed-system complexity (network failures, partial failures).
- Increased latency if decomposed badly (too many chatty calls).
- Data consistency across services is hard (no single DB transaction).
- Operational overhead (monitoring, tracing, deployment of many services).
- Bad decomposition → "distributed monolith" — services that are supposedly separate but actually tightly coupled: the worst of both worlds.

Microservices Design Patterns — organized by phase

The instructor groups patterns by the *phase* of building microservices. Learn them phase by phase:

1. Decomposition patterns — "how do I split the monolith?"

- Decompose by Business Capability — split along business functions (Orders, Payments, Inventory).
- Decompose by Subdomain (DDD) — split along Domain-Driven Design bounded contexts.
- Strangler Pattern ★ — *incrementally* migrate a monolith to microservices: put a façade/proxy in front, peel off one capability at a time into a new service, and route that traffic to it, until the monolith is "strangled" out. Safer than a big-bang rewrite.

2. Database patterns — "how does data live across services?"

- Database per Service — each service owns its schema/DB; others can't touch it directly (enforces loose coupling).
- Shared Database — multiple services share one DB (simpler, but couples services; anti-pattern for pure microservices).
- Saga Pattern ★★ — manage a transaction that spans multiple services via a sequence of local transactions; each has a compensating transaction to undo it on failure. Two styles: Choreography (services emit/react to events) and Orchestration (a central orchestrator drives the steps). (*Very common interview question — see 0024/0037.*)
- CQRS (Command Query Responsibility Segregation) — separate the write model from the read model (often separate stores), so reads and writes scale/optimize independently.
- API Composition — a composer service queries multiple services and joins the results in memory (used when data is split across services).
- Event Sourcing — persist state as an append-only log of events rather than current state; rebuild state by replaying events.

3. Communication patterns — "how do services talk?"

- Synchronous — REST/gRPC request-response (caller waits).
- Asynchronous — via a message broker/events (decoupled, resilient, but eventually consistent).

4. Integration patterns — "how do clients & services connect?"

- API Gateway — single entry point for clients (routing, auth, rate limiting, aggregation). (*See 0032.*)
- Aggregator — combine responses from several services.
- Chained / Chain of Responsibility — service A calls B calls C in a chain.
- Branch — invoke multiple chains in parallel and merge.

5. Observability patterns — "how do I see what's happening?"

- Log Aggregation (centralize logs), Distributed Tracing (trace a request across services), Health Check API, Metrics/Monitoring.

6. Cross-cutting / Deployment / Reliability

- Service Discovery (see 0038), Circuit Breaker (0042), Externalized Configuration, Sidecar / Service Mesh (0033), Blue-Green / Canary deployment.

Must-knows for interviews: Strangler (migration), Saga (distributed transactions), Database-per-service, API Gateway, CQRS.

0008 / 0009 — SCALE FROM 0 TO 1 MILLION USERS

The classic "evolve your architecture" question — taught as 9 incremental steps from a single server to a system handling 1M+ users.

Step 1 — Single server. Everything (web + app + DB) on one box. Client (web/mobile) hits the server directly. Fine for 0 users / a college project. No redundancy, no scale.

Step 2 — Separate the database from the app server. Move the DB onto its own machine so the web/app tier and data tier scale independently. Decide SQL vs NoSQL here (see 0015).

Step 3 — Add a Load Balancer + multiple web servers. Put a load balancer in front of a pool of *stateless* web servers. The LB distributes traffic, hides the servers behind a single virtual IP, and improves availability (if one server dies, traffic routes to the others). This is horizontal scaling of the web tier.

Step 4 — Database replication (Primary–Replica). One primary handles writes; multiple replicas serve reads. Benefits: higher read throughput, better availability, and failover (promote a replica if the primary dies). Reads scale out; writes still bottleneck on the primary.

Step 5 — Add a Cache. Introduce a cache layer (e.g., Redis/Memcached) in front of the DB for hot reads (cache-aside). Cuts DB load and latency. Watch: expiration (TTL), eviction policy (LRU), and cache consistency/invalidation.

Step 6 — CDN. Serve static content (images, CSS, JS, video) from edge PoPs geographically near users. Reduces latency and offloads the origin. Watch: TTL, invalidation, cost.

Step 7 — Make the web tier stateless. Move session state out of individual web servers into a shared store (e.g., Redis/DB). Now *any* server can serve *any* request → seamless horizontal scaling and auto-scaling. (Stateful servers break when the LB routes a user to a different server.)

Step 8 — Multiple Data Centers. Deploy across data centers in different regions with geo-routing / GeoDNS so users hit the nearest DC → lower latency + disaster recovery. Challenges: traffic redirection, data synchronization across DCs, and test/deploy across regions.

Step 9 — Message Queue + DB scaling + Observability.

- Decouple with a message queue — async processing (e.g., Kafka/RabbitMQ) so producers and consumers scale independently and spikes are absorbed.
- Scale the database via sharding — horizontal partitioning of data across shards when a single DB can't hold/serve the load.
- Add logging, metrics, monitoring, and automation (CI/CD) for operability.

Guiding principles

- Keep the web tier stateless → scale horizontally.
- Cache aggressively; use a CDN for static assets.
- Replicate for reads/HA, shard for write scale.
- Decouple heavy work behind queues.
- Add load balancing, multi-DC, and observability as you grow.

Problem it solves — naive hashing breaks on resize. To distribute keys across N servers, the naive approach is `server = hash(key) % N`. It works until N changes: add or remove a server and almost every key remaps to a different server → massive cache misses / data reshuffling. (This follows directly from the *horizontal sharding* problem in "Scale to 1M users.")

Recap — hashing: a hash function takes a variable-length key and returns a fixed-size value (e.g., `hash("shrayansh") = 143216...`). `% N` then buckets it. The fragility is entirely in the `% N`.

Consistent hashing — the idea

Map both servers and keys onto the same circular hash ring (hash space `0 ... 2n - 1`, wrapping around).

- Hash each server to a point on the ring.
- Hash each key to a point on the ring.
- A key is owned by the first server encountered clockwise from the key's position.

Why it's better

- Add a server: only the keys between the new node and its predecessor move — on average k/N keys (k = total keys, N = servers). Everything else stays put.
- Remove a server: only *its* keys move to the next clockwise node. No global reshuffle.

Virtual nodes (vnodes)

A single point per server causes uneven load (some arcs are much bigger) and hotspots. Fix: map each physical server to many points on the ring (virtual nodes). This smooths the distribution and, when a node leaves, spreads its load across many others instead of dumping it all on one neighbor.

Where it's used

Distributed caches, DynamoDB / Cassandra partitioning, load balancers, and sharded databases — anywhere you need to add/remove nodes without remapping the world.

CORE DESIGN QUESTIONS

0012 — DESIGN A URL SHORTENER

Goal: Build a TinyURL/Bit.ly-style service. A `POST(longURL)` returns a short URL and stores the mapping; a `GET(shortURL)` looks up and redirects to the long URL. (LinkedIn auto-shortens links in posts the same way.)

Phase 1 — Requirements & capacity

Key clarifying question: how short should the short URL be? Interviewer usually says "as short as possible." Derive the length from traffic:

- Assume 10 million URLs/day → over 100 years ≈ $365 \times 10M \times 100 \approx 365$ billion URLs to support.
- Allowed characters: $[0-9][a-z][A-Z] = 62$ characters.
- Combinations for length $L = 62^L$:
 - $62^6 \approx 56.8$ billion — *not enough*.
 - $62^7 \approx 3.5$ trillion — *enough*.
- ⇒ Use 7-character short codes.

Phase 2 — How to generate the short code

Option A — Hash function (MD5 / SHA-1): rejected.

- MD5 → 128-bit → 32 hex chars. SHA-1 → 160-bit → 40 hex chars.
- We need only 7 chars, so we'd have to truncate to the first 7 → high collision probability (different long URLs sharing the same 7-char prefix). ✗

Option B — Base-62 encoding of a unique ID: chosen.

- Take a unique numeric ID, convert it to base-62 (0–9, a–z, A–Z ↔ 0–61).
- Two sub-problems: (1) generating the unique ID, and (2) fixing the variable length.

The hard part — generating a unique ID in a distributed system

This is itself a classic interview question. Options, from naive to good:

Approach	How it works	Problem
Single DB auto-increment	One table's auto-increment column	Single point of failure; can't scale to 10M/day writes
Ticket Server	One centralized auto-increment service (Flickr-style)	Still a single point of failure; not scalable
Snowflake (Twitter)	64-bit ID = timestamp (41 bits) + machine ID + sequence number	Great, time-based, no central dependency; needs clock/coordination care
ZooKeeper (preferred here)	Distributed coordination service hands out ID ranges to workers	Slightly wasteful of ranges, but 3.5T total makes that irrelevant

ZooKeeper approach (recommended): ZooKeeper is *not* an ID generator — it's a distributed coordination service (Apache). Use it to divide the full `0 ... 3.5 trillion` space into ranges (e.g., 1M each) and assign one range to each worker/app server. Each worker mints IDs from *its* range only → guaranteed uniqueness with no cross-server synchronization. When a worker exhausts its range, ZooKeeper hands it a fresh unused range. Wasting a partially-used range is fine because we have 3.5T combos but only need 365B.

Fixing variable length (padding)

Small IDs produce short codes (ID 16 → base-62 `"g"`, 1 char). Pad to 7 characters. Can base-62 ever exceed 7 chars? No — the max ID in our range (`zzzzzzzz`) converts back to ≈3.5T, which is exactly our ceiling. As long as IDs stay within the range, the code is always ≤ 7 chars; pad the short ones.

Architecture

User → Load Balancer → TinyURL app servers (multiple data centers) → Cache + DB, with the app servers calling ZooKeeper for unique IDs.

- DB: a simple relational table (id, short_url, long_url) — one table, no complex relations, so SQL is fine.
 - Add a cache per data center for hot short-URLs.
 - No CDN needed — this is a lightweight lookup service. Interviewers are usually interested only up to the code-generation logic.
-

0013 — BACK-OF-THE-ENVELOPE ESTIMATION

Why it matters: If you jump straight into a design ("I'll add a load balancer, CDN, cache..."), the interviewer will ask *"Do you actually need them? How many servers? How much cache?"* Estimation drives your design decisions with numbers and shows you won't over- or under-provision.

Ground rules

- Rough / "T-shirt size" numbers — high-level, not accurate. Your Facebook estimate won't match real Facebook, and that's fine.
- Don't spend > 10 minutes. 80–98% of the time it doesn't materially change the (already scalable) design, and interviewers often skip it — *ask first if they even want it*.
- Keep assumptions simple — use multiples of 10 (10M, 100M, 1B), never "435 million."

Cheat sheets to memorize

Powers of 10 (every 3 zeros):

Thousand (10^3) → Million (10^6) → Billion (10^9) → Trillion (10^{12})

Storage units (every 3 zeros): KB → MB → GB → TB → PB

Data-size assumptions:

- 1 character = 2 bytes (Unicode; ASCII would be 1 byte)
- long / double = 8 bytes
- 1 image ≈ 300 KB

Killer formula: $X \text{ million users} \times Y \text{ (unit)}$ → count the zeros, add them, map to the storage unit.

- $X \text{ million} \times Y \text{ MB} = XY$ with $6+6 = 12$ zeros → XY TB
- e.g., $5 \text{ million} \times 2 \text{ KB} = 10$ with $6+3 = 9$ zeros → 10 GB

What to compute

Typically three things: # of servers, RAM (cache), storage — plus a CAP trade-off statement at the end.

Worked example — estimate Facebook

1) Traffic

- Total users = 1 billion; DAU = 25% = 250 million.

- Each user: ~ 5 reads + 2 writes ≈ 7 queries/day.
- Seconds/day $\approx 86,400 \rightarrow$ round to 100,000.
- QPS = $\frac{250M \times 7}{100,000} \approx 18,000$ queries/sec.

2) Storage

- 2 posts/user/day, 250 chars/post $\times$ 2 bytes = 500 B/post $\rightarrow$ 1 KB for 2 posts.
- Posts/day = $\frac{250M \times 1 \text{ KB}}{1} = 90\text{-zeros logic} \rightarrow 250 \text{ GB/day}$.
- 10% of DAU (25M) upload 1 image (300 KB): $\frac{25M \times 300 \text{ KB}}{1} \approx 7\text{--}8 \text{ TB/day}$.
- Store for 5 years (~ 2000 days): posts $\approx 500 \text{ TB}$, images $\approx 16 \text{ PB}$.

3) RAM (cache)

- Cache last 5 posts/user: $\frac{5 \times 500 \text{ B}}{1} = 2500 \text{ B} \approx 3 \text{ KB/user}$.
- $\frac{250M \times 3 \text{ KB}}{1} \approx 750 \text{ GB total cache}$. If one machine holds 75 GB $\rightarrow$ 10 cache machines.

of servers (from QPS + latency)

- Target latency: 95% of requests in 500 ms $\Rightarrow$ 1 thread serves 2 req/sec; 50 threads/server $\Rightarrow$ 100 req/sec/server.
- $\frac{18,000}{100} \approx 180$ servers.

4) CAP trade-off

- For Facebook: choose AP (availability + partition tolerance), drop strong consistency — a slightly stale feed is acceptable; downtime is not.

0014 — DESIGN A KEY-VALUE STORE (DYNAMODB-STYLE)

★ The single most information-dense topic in the playlist. Models Amazon DynamoDB (used in Amazon's Add-to-Cart). Covers the internals of a distributed, decentralized, highly-available KV store.

3 Goals

1. Scalability 2. Decentralization 3. Eventual consistency

Achieved via 6 building blocks: (1) Partition, (2) Replication, (3) Get/Put, (4) Data Versioning, (5) Gossip Protocol, (6) Merkle Tree.

1) Partition $\rightarrow$ **Scalability**

A single hash table can't hold billions of users' data. Use consistent hashing: place servers on a ring, assign each a key range (S1: 1–50, S2: 51–100, ...). A key is hashed $\rightarrow$ falls in a range $\rightarrow$ owned by that server. Virtual nodes spread each physical server across many ring positions to avoid hot-spots and uneven load.

2) Replication $\rightarrow$ **Decentralization** (no single point of failure)

If S1 (which owns key 45 = "car") dies, its data is lost unless replicated.

- N = replication factor (default 3, configurable).
- The server owning the key is the coordinator. It copies the value to the next $N-1$ servers clockwise on the ring.
- Copies skip virtual nodes of the same physical server and often prefer servers in different data centers (survive a DC fire) — so replicas aren't strictly the sequential next nodes.
- Each key-range has a preference list: `[coordinator, replica1, replica2, ...]`. Every node knows every range's preference list (via gossip). The preference list is central to get/put.

3) Get / Put — Quorum: $R + W > N$

- Load balancer types:
 - Generic LB → request lands on *any* node; if it's not the coordinator, it hops to the coordinator (per the preference list). Simple, but higher latency.
 - Partition-aware LB → routes directly to the coordinator. Lower latency, more logic in the LB.
 - If the coordinator is down, the next replica in the preference list serves.
- PUT: coordinator writes locally, then asynchronously replicates to $N-1$ replicas. It returns success once W replicas acknowledge.
- GET: coordinator asks all replicas for their copy and returns once R replicas respond.
- The rule $R + W > N$ guarantees the read set and write set overlap (strong-ish consistency knob):
 - $W=1$ → fast writes; $R=N$ → strong reads.
 - Tune R and W to trade latency vs consistency.

4) Data Versioning → **Vector Clocks** (resolve conflicts)

Why read from multiple replicas? Because network failures cause replicas to diverge ($S1="car"$, $S2="cart"$, $S3="carm"$ for the *same* key). Concurrent writes hitting different coordinators (when some nodes are down/partitioned) create multiple versions.

- A vector clock = list of `[server, counter]` pairs per object.
- On each write, the handling server bumps *its* counter. If one clock is an ancestor of another (all counters $\leq$), the newer one wins automatically.
- If clocks conflict (neither dominates), the coordinator returns all conflicting versions to the client. The client resolves (e.g., Last-Write-Wins, or app-specific merge — DynamoDB *cart merges* items) and writes back the reconciled version.
- This is why the store is eventually consistent: a read may return stale data, but repeated reads (after reconciliation) converge.
- CAP-wise: DynamoDB/KV stores choose AP — sacrifice C for availability.

5) Gossip Protocol → membership & failure detection

How does every node know every range/preference list and who's alive?

- Each node keeps a membership list and periodically (e.g., every 1 s) sends a heartbeat + metadata (ranges it owns) to random nodes, which propagate it onward.
- Each node tracks the last-heard counter/time for others. If a node's heartbeat goes stale and multiple nodes agree it's silent, it's marked down and removed. (One node's opinion isn't enough — avoids false

positives.)

6) Merkle Tree → efficient replica anti-entropy

To check whether a replica holds the latest data for a range without comparing millions of keys one by one:

- Build a Merkle tree per range: keys at the leaves, each parent = hash of its children, up to a root hash.
- Compare roots between coordinator and replica: equal roots ⇒ identical data, stop. If unequal, descend only into the subtree whose hashes differ — narrowing to the exact out-of-sync key range in $O(\log n)$ comparisons.

Interview gold: consistent hashing + virtual nodes, coordinator + preference list + N/R/W quorum, vector clocks, gossip, Merkle trees — one question that teaches half of distributed systems.

0015 — SQL VS NOSQL — WHICH DATABASE TO CHOOSE

Why it matters: In an HLD round, answering "we can use anything" or picking one without justification is a red flag. You must reason from the data and access patterns. Compare across 4 dimensions: Structure, Nature, Scalability, Property.

SQL (Relational / RDBMS)

Dimension	SQL
Structure	Tables → rows & columns, relations between tables, predetermined (fixed) schema — you define the schema <i>before</i> inserting.
Nature	Centralized/concentrated — a given entity's data across all its tables lives on one server (barring manual sharding).
Scalability	Scales vertically best (bigger RAM/CPU/disk). Horizontal sharding exists but is not well supported.
Property	ACID — Atomicity, Consistency, Isolation, Durability → guarantees data integrity / consistency.

NoSQL (Non-relational / "Not only SQL")

4 structure types:

- Key-Value (e.g., DynamoDB, Redis) — value is opaque; you can query only by key → very fast.
- Document (e.g., MongoDB) — value is JSON/XML; you can query on the value too.
- Column-wise (e.g., Cassandra) — each key maps to a dynamic list of column:value pairs (different keys can have different columns).
- Graph (e.g., Neo4j) — data as nodes + edges (relationships); great for direct relationship traversal.

Dimension	NoSQL
Structure	Unstructured; one of KV / Document / Column / Graph (above).
Nature	Distributed — data spread across many nodes (10M records split across nodes 2M each).
Scalability	Scales horizontally easily (add nodes, shard).
Property	BASE — Basically Available, Soft state, Eventual consistency.

BASE explained: *Basically Available* = highly available via replication across distributed nodes. *Soft state* = state may change over time without input (nodes sync via vector clocks). *Eventual consistency* = a read may be stale, but converges after replicas sync.

When to use which — decision factors

Factor	Choose SQL	Choose NoSQL
Query flexibility	Complex/ad-hoc joins, changing query needs	Only basic key-based lookups / known access pattern
Data shape	Highly relational (parent-child hierarchies)	Loosely related / non-relational
Data integrity	Must not lose a transaction (banking, finance)	Losing occasional data among billions is OK
Availability & scale	Moderate	Need high availability + high write/search throughput on big data, can tolerate some inconsistency

One-liner: SQL = consistency + relations + flexible queries, scales vertically. NoSQL = availability + massive scale + fast simple queries, scales horizontally, eventual consistency.

0016 — DESIGN A CHAT APPLICATION (WHATSAPP / MESSENGER)

Covers WhatsApp / Discord / Telegram / Slack / FB Messenger. Aim for a real-time, scalable, available messaging system.

Requirements

Functional: 1:1 send/receive (text first; images/files extendable), group messaging, last-seen / online-offline, login/authentication.

Non-functional: scalability (billions of msgs/day), low latency (near-real-time delivery), availability.

Capacity (quick)

2B users, 50M DAU, 10 msgs to 4 people = ~2B msgs/day. 1 msg ≈ 100 bytes → ~200 GB/day. Store 10 yr history → TB/PB. (Note: WhatsApp stores chat on the device; Discord/Telegram/Slack/Messenger keep it server-side.)

Core: connection protocol

- Peer-to-peer (users talk directly by IP) → not scalable, can't do history/grouping ⇒ use a central chat server (client-server).
- Sending via HTTP works (client initiates), but receiving fails — HTTP is request/response; the server can't initiate a push to the client.
- Options to let the server push:

Technique	How	Verdict
Polling	Client repeatedly asks "any messages?"; connection opens/closes each time	Wasteful (mostly "no"); not scalable
Long polling (a.k.a. pushing)	Server holds the request up to a threshold (e.g., 1 min) and replies when a message arrives or timeout	Better, but still blocks threads at scale
WebSocket 	Bidirectional, persistent connection (handshake once, stays open)	The right choice — server can push anytime

⇒ Clients connect to chat servers over WebSocket.

1:1 messaging architecture

Problem: User1 is on `chat-server-100`, User2 on `chat-server-500`. When User1 sends to User2, how does server 100 find server 500?

- User Mapping Service (implemented with ZooKeeper) maintains `user → chat-server` mappings and assigns a (geographically near) chat server when a user comes online.
- Flow: User1 → WebSocket → chat-server-100 → asks User-Mapping "where is User2?" → gets chat-server-500 → forwards message → server 500 pushes to User2 over its WebSocket.

Database choice

Read ops (open a chat, group history, member list, profile) have no complex joins; need low-latency search over huge history + high availability. ⇒ NoSQL, column-wise (Cassandra) — as Discord/Facebook use.

- 1:1 table: `messageId, from, to, timestamp`. Partition key = (from, to) → all of a conversation's history lands on one node. messageId provides ordering within a partition (use timestamp or a local ID generator — no global Snowflake needed, since ordering is per-partition).
- Group table: `groupId, from, message, timestamp`. Partition key = groupId; messageId orders within the partition.

Offline handling

If User2's chat server is down / User2 has no connectivity, its ZooKeeper entry is removed. User1's message can't be delivered → persist to DB as unread. When User2 logs back in (HTTP → User-Mapping assigns a new chat server), that server checks the DB for unread messages and pushes them.

Group messaging

Chat server asks the Group Service for the group's members, then asks ZooKeeper which chat server each member is on, and fans the message out to those servers, which deliver over WebSocket. A separate Group Management Service (HTTP + its own DB) handles create/delete/join/add-members.

Last-seen / online-offline: Presence System

- Client sends a heartbeat every few seconds; the Presence Service records the last-heartbeat time per user.
- If no heartbeat for a threshold (e.g., 1 minute) → mark offline.
- Why a threshold (not just ZooKeeper presence)? The "train tunnel" problem — connectivity flapping every 1–2 s would flip status online/offline rapidly (bad UX). The threshold debounces this.

Full component list

Login/Auth Service (HTTP + DB) · User Mapping Service (ZooKeeper) · Chat Servers (WebSocket) · NoSQL message store (Cassandra) · Group Service · Presence Service — all behind a Load Balancer for HTTP traffic.

DISTRIBUTED SYSTEMS BUILDING BLOCKS

0017 — DESIGN A RATE LIMITER

Why it matters: Protects APIs from DDoS — an attacker floods the server with requests, exhausting limited RAM/disk, taking it down so genuine users get rejected. As a backend engineer you must know how to rate-limit APIs. Rejected requests return HTTP 429 (Too Many Requests).

You must know 5 algorithms before designing:

1) Token Bucket

- A bucket holds up to capacity tokens; a refiller adds R tokens every T (config-driven, dynamic).
- Each request consumes 1 token; if none available → denied (429). Excess tokens beyond capacity overflow (discarded).
- Easily implemented per-user/per-API with a counter + timestamp. Rule example: "each user, 3 tokens/min for POST."
-  Allows bursts up to capacity. Used by Amazon, Stripe.

2) Leaking Bucket

- A queue (FIFO) with fixed capacity; requests are processed at a constant rate (the "leak").
- Queue full → request denied (429).
-  Smooths output to a constant rate.  Bad for bursty legitimate traffic (e.g., Amazon Prime evening spike): old requests sit in the queue while new ones are dropped. Ask the interviewer whether a constant rate fits the app.

3) Fixed Window Counter

- Divide time into fixed windows (e.g., 5-min buckets); each window gets a counter (e.g., 3). Requests decrement it; 0 → denied.
-  Edge/boundary problem: 3 requests at the *end* of window 1 and 3 at the *start* of window 2 = 6 requests within one 5-min span, double the intended limit.

4) Sliding Window Log

- Fix the fixed-window edge problem. Store a log of timestamps (not a counter). On each request, drop timestamps older than the window, then check if `log size < limit`.

- If within limit → log the new timestamp and allow; else deny.
-  Accurate (no boundary spikes).  Memory-heavy — stores a timestamp per request even for rejected ones.

5) Sliding Window Counter

- Hybrid of fixed-window + sliding-window log. Keep per-window counters and compute a weighted count across the current and previous window based on overlap:

count ≈ current_window_count + previous_window_count × (overlap % of previous window).
-  Smooths boundary spikes with far less memory than the log. A practical middle ground (used by Cloudflare).

Configuration

Rules are config-driven and typically per-user, per-API, per-time-window (e.g., user X → 3 POST/min). Store counters/timestamps keyed by user+API.

Algorithm	Data structure	Bursts	Memory	Boundary-accurate
Token Bucket	counter	 allowed	low	—
Leaking Bucket	queue	 smoothed out	low	—
Fixed Window Counter	counter		low	 (2× spike)
Sliding Window Log	timestamp log		high	
Sliding Window Counter	counters		low	 (approx)

0018 — IDEMPOTENCY HANDLING

Idempotency vs Concurrency (don't confuse them):

- Concurrency: *one* resource, *many* users contend for it (e.g., many people booking the same movie seat).
- Idempotency: handling duplicate requests — a client can safely retry an operation without side effects (retry N times → exactly one DB row).

Which HTTP methods are idempotent by default?

- GET, PUT, DELETE — already idempotent. (GET has no side effect; PUT "set name = X" repeated leaves the same state; DELETE repeated leaves it deleted.)
- POST — NOT idempotent — each call creates a new row/resource. Duplicate POSTs = duplicate payments / duplicate cart items. We must make POST idempotent.

How duplicates arise (two cases):

- Sequential: client POSTs → server processes but times out on the client side → server actually completes → client retries → duplicate.
- Parallel: two identical POSTs arrive at the same time (e.g., double browser, possibly different servers).

Solution — Idempotency Key

An idempotency key = a unique value (UUID, optionally + operation + timestamp). Two agreements with the client: (1) the client generates the key; (2) a new key per distinct operation. Client sets it in the request header.

Server flow:

1. Validate key present in header; if not → HTTP 400 (validation error).
2. Read the key from an idempotency DB.
3. If absent → insert `(key, status = CREATED)`, run the operation; on success → mark CONSUMED, return HTTP 201 (created).
4. If present → check status:
 - CONSUMED → the original already finished → HTTP 200 (already done, no new resource).
 - CREATED (still in progress) → HTTP 409 Conflict (a duplicate is mid-flight; retry later).

Idempotency-key lifecycle: `CREATED / ACQUIRED → CONSUMED / CLAIMED` (once consumed, never reused).

Handling the parallel case — Mutual Exclusion

Two parallel requests can both read "key absent" and both create resources. Fix: wrap the critical section (read-key → write-key) in a lock (mutex / semaphore / `synchronized`) so only one request enters at a time. The first creates + consumes; the second then sees CONSUMED → returns 200.

Follow-up — multiple clusters / separate DBs

If duplicates hit different servers with different DBs, DB-to-DB sync is too slow (minutes) to catch the duplicate in time. Use a shared cache for the idempotency keys/lock — cache synchronization is near-real-time (ms).

0019 — HIGH AVAILABILITY ARCHITECTURE

Asked many ways (all the same underlying question): "design high-availability / data-resilience architecture," "achieve 99.999% (five-nines) availability," "avoid a single point of failure," "explain active-passive vs active-active." Resiliency = ability to recover from failure.

The problem — single node

`Client → LB → microservices → single primary DB`. If the DB dies, all reads and writes fail — no five-nines, a single point of failure, no resilience (recovery could take hours/days).

Multi-node solution

Every company keeps ≥ 2 data centers (e.g., Mumbai + Pune).

Active-Passive (Active-Standby)

- Only one DB is primary (a.k.a. live / read-write); the others are replicas in a Disaster Recovery (DR) data center.
- Writes always go to the primary (because Oracle/MySQL/Postgres are not multi-master — only one DB can accept writes). Reads can be served by the read-only replicas (improvement over routing everything to primary).

- Requests hitting the DR data center route their writes to the primary; the primary syncs (one-directional) to replicas.
- Failover: if the primary dies, promote a DR replica to primary and reroute traffic → survives the failure (no SPOF).
- Disadvantages: (1) latency add-on when a request lands on the DR DC and must reach the far primary; (2) a failover gap (10–15 min) during which writes fail while promotion happens; (3) doesn't scale for write-heavy apps (all writes funnel to one primary).

Active-Active

- Multiple live/primary DBs, each serving both reads and writes, with bidirectional sync. Requires a multi-master DB — Cassandra / most NoSQL support this (Oracle/MySQL/Postgres generally don't).
-  Full resource utilization across all DCs → handles more traffic, both read and write scale.
-  Hard part = synchronization: concurrent writes to the same row in two DCs → conflicts needing conflict resolution; and a write in DC1 may not have synced before a read in DC2 (stale reads). This is the core complexity of active-active.

	Active-Passive	Active-Active
DBs accepting writes	1 (primary)	Many (multi-master)
Sync	one-directional (primary→replicas)	bidirectional
DB type	Oracle/MySQL/Postgres	Cassandra / NoSQL
Resource use	replicas underused	fully utilized
Write scaling	 funnels to one	
Main challenge	failover gap, latency	conflict resolution

0020 — DISTRIBUTED MESSAGING QUEUE (KAFKA / RABBITMQ)

Basics: Producer → queue → Consumer. Why needed / advantages:

1. Asynchronous processing — e.g., e-commerce order emits "send notification"; the user doesn't wait. Lowers latency.
2. Retry capability — if the consumer (e.g., notification service) is down, the message stays/re-queues for retry.
3. Pace matching (buffering) — many producers emitting faster (30/s) than a consumer can handle (15/s); the queue absorbs the burst and the consumer drains at its own pace (e.g., thousands of cabs sending GPS every 10 s → a dashboard consumer can't keep up in real time).

Point-to-Point vs Pub-Sub:

- Point-to-Point: a message is consumed by exactly one consumer.
- Pub-Sub: an exchange broadcasts each message to multiple queues, so many consumers each get it.

Kafka — components

Producer → Broker (Kafka server) → Topic → Partition → Offset; Consumer ∈ Consumer Group; Cluster = group of brokers; ZooKeeper coordinates.

- Topic = logical channel; a placeholder for partitions (can have many).
- Partition = where messages actually live (an append-only, ordered log with offsets 0,1,2...). Partitions of one topic can live on different brokers.
- Offset = position in a partition.
- Broker = one Kafka server; a Cluster = many brokers on different machines.
- ZooKeeper = coordination — tracks which topic/partition lives on which broker, plus consumer offsets.

Message format: { key, value, partition?, topic } — value = actual data; topic mandatory.

Partition selection: (1) if key present → hash(key) picks the partition (same key → same partition → ordering per key); (2) else if a partition is specified → use it; (3) else round-robin across partitions.

Consumer groups & offsets (the important part)

- Each consumer belongs to a consumer group. Within a group, each partition is read by exactly one consumer (two consumers in the same group can't read the same partition). Different groups can each read the same partition independently.
- Committed offset (tracked in ZooKeeper per group+consumer+topic+partition) marks how far a consumer has successfully read.
- Consumer failover: if a consumer dies, another free consumer in the same group takes over its partition and resumes from the last committed offset — the reason consumer groups exist.

Replication (leader/follower)

- Each partition has a leader (on one broker) and followers (replicas on other brokers). All reads/writes go through the leader; followers continuously sync from it.
- If the leader broker dies, a follower is promoted to leader → no message loss.

Failure handling

- Queue size limit reached → add more brokers (scale out).
- Broker/queue down → follower becomes leader; messages preserved.
- Consumer down → another group member resumes from committed offset.
- Consumer can't process a message (buggy message) → don't advance the offset; retry N times (config); after retries exhausted, move the message to a Dead Letter Queue (DLQ) / failure queue and advance. Someone fixes and re-injects it later.

Kafka vs RabbitMQ

	Kafka	RabbitMQ
Delivery	Pull (consumer polls)	Push (broker pushes to consumer)
Routing	partitions via key/round-robin	Exchange + routing keys → queues
Offset	yes (committed offset)	no offset; failed msg is re-queued (retry → DLQ)

RabbitMQ exchanges:

- Fanout — broadcast to all bound queues.
 - Direct — routing key must exactly match the binding key → one queue.
 - Topic — wildcard matching (e.g., `*.123`) → routed to matching queues.
-

0021 — PROXY SERVERS (FORWARD & REVERSE)

Analogy: A child wants chocolate but asks mom, who goes to the shop and brings it. The child (client) never talks to the shopkeeper (server) directly — mom is the proxy. A proxy sits between client and server; all requests pass through it; one proxy can serve many clients.

Two types — they differ mainly in direction:

Forward Proxy (a.k.a. "simple proxy" — protects the **client**)

Sits in front of a group of clients (intranet) → goes out to the internet on their behalf. Hides the client network — servers only see the proxy's IP, never the client's.

5 advantages:

1. Anonymity — client IP/location hidden from servers.
2. Request grouping — clubs identical requests (100 clients asking google.com) into fewer outbound calls.
3. Access restricted content — bypass geo-blocks by appearing to come from another location.
4. Security / access control — enforce rules on what clients may access (e.g., block facebook.com).
5. Caching — cache static content; serve repeats from cache without going out.

Disadvantage: works at the application layer → must be set up per application.

Reverse Proxy (protects the **server**)

Opposite direction — internet requests hit the reverse proxy, which forwards to the appropriate backend server. The outside world never sees the server's IP.

Advantages:

1. Security / DDoS protection — attackers can only hit the reverse proxy, not the origin; the proxy has resources to absorb attacks.
2. Caching — serve cached responses without touching the origin.
3. Reduced latency — placed near users (edge).
4. Load balancing — distribute requests across multiple backend servers.

CDN is a reverse proxy — edge nodes in Paris/US/India cache content near users; only cache misses reach the origin (e.g., in Singapore). You'll see CDNs in every "design YouTube/Facebook" answer.

Key distinctions (very common interview traps)

Comparison	Difference
Proxy vs VPN	Proxy gives IP anonymity + caching + logging but no encryption. VPN creates an encrypted tunnel (encrypt at client, decrypt at VPN server) — much more than a proxy.
Reverse proxy vs Load balancer	A reverse proxy can act as a load balancer, but an LB can't be a proxy (no anonymity/caching/logging). LB is needed only with >1 server; a reverse proxy is useful even with one server (caching, anonymity, logging).
Proxy vs Firewall	Firewall does packet scanning (header, ports, source/dest IP) at the packet level. Proxy works at the application level (can inspect data, add data-based rules). Modern "proxy firewalls" can block too, but at the application layer.

One-liner: Forward proxy protects clients; reverse proxy protects servers.

0022 — LOAD BALANCERS

Purpose: Distribute incoming traffic across servers so no single server is overloaded (also does logging, caching, etc., but distribution is the core job).

L4 vs L7

	L4 — Network LB (transport layer)	L7 — Application LB (application layer)
Routes on	TCP/UDP port, source/dest IP	Header, session, cookies, data, response
Caching	✗	✓ (can read responses)
Speed vs power	Faster	More advanced/smarter

Static algorithms (no runtime computation)

- 1. Round Robin — cycle servers in order. ✓ Simple, equal distribution. ✗ Ignores server capacity — a weak server gets the same share as a strong one and may go down.
- 2. Weighted Round Robin — assign weights = capacity; a 3× server gets 3× the requests. ✓ Protects low-capacity servers, static (no computation). ✗ Ignores request processing time — a weak server can still get stuck with a few very heavy requests.
- 3. IP Hash — `hash(source IP)` → server. ✓ Same client always routes to the same server (sticky). ✗ If clients sit behind a forward proxy, they share one source IP → all hash to one server (overload); also no guarantee of even distribution.

Dynamic algorithms (computed at runtime)

- 4. Least Connections — route to the server with the fewest active connections. ✓ Adapts to load. ✗ An active TCP connection may carry no/low traffic, so "fewest connections" ≠ "least loaded."
- 5. Weighted Least Connections — route by the min ratio of active-connections / weight. Combines load + capacity.
- 6. Least Response Time — route to `min(active_connections × TTFB)`, where TTFB = Time To First Byte (interval between sending a request and receiving the first response byte, measured dynamically). Ties → fall back to round robin. ✓ Accounts for both concurrency and real server responsiveness.

0023 — CACHING

Definition: Store frequently used data in fast-access memory (RAM/Redis) instead of fetching every time from slow storage (disk/DB). Makes the system faster (lower latency) and can add fault tolerance (see write-back).

Cache lives at every layer: browser cache → CDN (static content) → load balancer → server-side application cache (Redis) ← focus here → DB. Server-side cache sits between app and DB: the app checks the cache before hitting the DB.

Distributed caching

One cache server → limited scalability + single point of failure. Instead use a cache pool of many cache servers; each app server has a cache client that picks a cache server via consistent hashing (place cache servers on a ring; a key maps clockwise to the first server).

The 5 caching strategies

Strategy	Read path	Write path	Pros	Cons
Cache-Aside (lazy)	App checks cache; miss → app reads DB, app writes to cache	writes go straight to DB (cache untouched)	Good for read-heavy; survives cache down (falls back to DB); cache structure independent of DB schema	New data always misses first; cache-DB inconsistency if writes don't update/invalidate cache
Read-Through	App checks cache; miss → the cache library reads DB and fills itself	writes to DB separately	Read-heavy; fetch/fill logic separated from app	New data misses; inconsistency without a write strategy; cache doc must mirror DB schema (1:1)
Write-Around	(paired with cache-aside/read-through)	Write directly to DB, invalidate/mark dirty the cache entry	Fixes read inconsistency; read-heavy	New-data reads still miss; write fails if DB down (not fault-tolerant); useless alone
Write-Through	(paired with a read strategy)	Write to cache first, then synchronously to DB (both succeed or roll back)	Cache & DB always consistent; new data is cached → more cache hits	Adds write latency; needs two-phase commit (rollback on partial failure); still fails if DB/cache down
Write-Back / Write-Behind	(paired with a read strategy)	Write to cache first, then asynchronously to DB (via a queue)	Best for write-heavy; lowest write latency; fault-tolerant — system stays up even if DB is down for hours	Risk of data loss / gaps: if the cache entry's TTL expires before the queue flushes to a still-down DB, data vanishes from both

Fault tolerance insight: Only write-back makes writes survive a DB outage (the cache + queue absorb it) — this is the "how does caching give fault tolerance?" answer.

0024 — DISTRIBUTED TRANSACTIONS

Frequently asked at all experience levels. A transaction = a group of operations run against the DB (e.g., debit A ₹100 and credit B ₹100).

ACID recap

- Atomicity — all operations succeed or all roll back.
- Consistency — DB is in a valid state before and after the transaction.
- Isolation — concurrent transactions appear to run serially (via row locks).
- Durability — once committed, data survives even a DB crash.

The problem — transactions are **local**

On one DB, `BEGIN → lock rows → update → COMMIT (or ROLLBACK)` works because all operations target that DB's transaction manager. But in a distributed system (e.g., Order DB + Inventory DB), each DB has its own transaction manager. If the order update commits but the inventory update fails, the order can't be rolled back by the inventory's transaction — they're separate. Need a cross-DB protocol.

Three solutions

1) Two-Phase Commit (2PC) — synchronous, most popular.

Needs a Transaction Coordinator that supports all participants (microservices).

- Phase 1 — Prepare / Voting: coordinator sends each participant the update; each locks rows, applies changes (not committed), and votes "prepared / OK" or "no."
- Phase 2 — Decision / Commit: if all vote OK → coordinator sends COMMIT; if any vote no → sends ABORT to all.
- Everyone writes to a persistent log file before each step, so a recovering node can look up what happened.
- Failure cases (coordinator or participant fails → messages lost):
 - Prepare message lost → participant times out and aborts (safe; if a late prepare arrives, it replies "no").
 - OK message lost → coordinator times out and aborts; a recovering participant asks the coordinator, which says "abort."
 - Commit message lost / coordinator down → **✗ BLOCKING** — the participant has voted OK but doesn't know the decision; it must wait for the coordinator to recover and can't decide on its own. This blocking flaw is 2PC's main weakness.

2) Three-Phase Commit (3PC) — improves 2PC by splitting the decision phase, non-blocking, but complex (rarely used).

- Phase 1 — Prepare (same as 2PC).
- Phase 2 — Pre-Commit: coordinator shares its decision (commit/abort) with all participants without executing it — "here's what I decided, in case I go down."
- Phase 3 — Commit: participants actually commit/abort.
- Why non-blocking: if the coordinator dies after pre-commit, participants already know the decision (from their logs) and proceed on their own. If it dies before pre-commit, participants can query each other ("did you get pre-commit?"); if none did, they safely abort. (2PC and 3PC both let participants know about each other.)

3) Saga Pattern — asynchronous, for long-running transactions.

Used when a transaction spans many participants sequentially ($P1 \rightarrow P2 \rightarrow P3 \rightarrow \dots$) and holding locks across all of them (as 2PC/3PC do) is infeasible.

- Each participant commits its own local transaction, then triggers the next (often by publishing an event).
 - On failure at step N, each prior step runs a compensating transaction to undo its work, propagated backward via events (choreography) — e.g., P5 fails → emits event → P4 rolls back → P3 rolls back → ...
 - 2PC/3PC are synchronous (user waits, locks held); Saga is asynchronous (local commits + compensations via a queue). Saga is the go-to for distributed, long-lived business transactions.
-

0025 — DATABASE INDEXING

★ A frequently-asked deep-dive (focus: RDBMS). To understand it you must first know (1) how rows are physically stored, (2) the index types, (3) the data structure (B+ tree).

1) How table data is actually stored

The table you see is a logical representation, *not* physical.

- Data Page (managed by DBMS, typically 8 KB): the unit DBMS stores rows in. Layout of an 8 KB (8192-byte) page:
 - Header (~96 bytes): page number, free space, checksum, etc.
 - Data records (~8060 bytes): the actual rows.
 - Offset array (~36 bytes): an array of pointers, each pointing to a row in the data-records area. (Its ordering matters for clustered indexes — see below.)
 - So if a row is 64 bytes → $8060 / 64 \approx 125$ rows per page. A table with many rows spans many data pages (page 1, 2, ... 100).
- Data Block (managed by the storage system, *not* DBMS; 4–32 KB, commonly 8 KB): the minimum unit of a single I/O read/write on disk. DBMS has no control over which block a page lands in (blocks can be scattered). If block size > page size, one block holds multiple pages.
- DBMS maintains a page → block mapping (since it doesn't control block placement), so it knows which physical block holds each data page.

2) Why indexing — and the data structure

Without an index, finding "employee ID 35" means scanning every page and every row → $O(N)$. An index makes lookups fast using a B+ tree → $O(\log N)$ for search/insert/delete. ("B" = Balanced.)

B-tree recap (order M = at most M children, M–1 keys per node):

- Keeps data sorted; all leaves at the same level.
- Left pointer of a key → children < key; right pointer → children ≥ key.
- Insertion: add in sorted order; if a node overflows (3 keys in an order-3 tree), push the middle key up to the parent and split — recursively, growing the tree upward. This keeps it balanced.

B+ tree = B-tree + leaf nodes linked together (a linked list across leaves → fast range scans). In DBMS:

- Root / intermediate nodes hold values used only for navigation (a value there may even be deleted from the DB — it just guides the search).
- Leaf nodes hold the actual indexed column values (plus a pointer to the data page holding that row).

3) Connecting the dots — insert flow

When a row is inserted (say indexed on employee ID):

1. Insert the key into the B+ tree to find its correct sorted position.
2. Check the neighbor leaf to see which data page nearby rows live in, and try to place the row there.
3. Load that page (via page→block mapping), and if it has space, insert the row and store a pointer (key → data page).
4. If the page is full → page split: create a new page, split the rows across the two pages, and update all affected pointers (some neighbors' keys now point to the new page) and the page→block mapping.

Index types

Clustered Index — *the order of rows inside the data pages matches the order of the index.*

- Achieved via the offset array: even if rows were inserted in jumbled physical order (1,4,5,2), the offset array's pointers are arranged so traversing them yields index order (1,2,4,5). The physical rows don't move; the offset ordering does.
- Only ONE clustered index per table (you can order rows by only one key).
- Which column? Priority = Primary Key (unique + not null). If none is defined, DBMS creates a hidden auto-increment column and uses it as the clustered key.
- ⚠ Adding a primary key later forces DBMS to rebuild the B+ tree, reshuffle pages, and update all pointers — expensive.

Non-Clustered (Secondary) Index — for secondary/composite indexes (e.g., index on `name`).

- Builds a separate B+ tree whose leaves point to the data pages, but does NOT reorder the rows/pages.
- Many allowed per table (unlike clustered).
- Stored in index pages (themselves mapped to data blocks).

The cost of indexing (why not index everything)

Each secondary index = another B+ tree consuming memory (index pages on disk + their block mapping). For 1M rows × 3 indexes = 3 large trees. Every insert/update/delete must update the clustered index and all secondary indexes (page splits ripple through pointers; deletes must remove leaf nodes). ⇒ Index deliberately — the read speedup is paid for in write overhead and storage.

Search flow with an index

1. Load the index pages (from their data blocks).
 2. Traverse the B+ tree to find which data page holds the key (e.g., ID 35).
 3. Use the page → block mapping to find the data block.
 4. Load only that block into memory and read the row — no full scan, even with millions of blocks.
-

CONCURRENCY & SECURITY

0026 — CONCURRENCY CONTROL IN DISTRIBUTED SYSTEMS

★ Asked in both LLD (e.g., "how do you handle concurrency in BookMyShow/parking lot?") and HLD ("explain distributed concurrency control").

The problem: Three users try to book the same seat at the same time. All three read the row (status = free), all set it to "booked," all get success → seat sold three times. The booking code is the critical section (logic accessing a shared resource).

`synchronized` isn't enough: A `synchronized` block serializes threads within one process. In a distributed system, the service runs on multiple machines/processes behind a load balancer, so three requests hit three processes — `synchronized` can't coordinate across them. You need distributed concurrency control → Optimistic or Pessimistic (correct term: *concurrency control*, though everyone says "optimistic/pessimistic locking").

Prerequisite 1 — Transactions

A transaction gives integrity: if any statement fails, roll back all succeeded statements so the DB returns to a consistent state (debit A ₹20 succeeds but credit B fails → roll back the debit).

Prerequisite 2 — DB Locking

Locks ensure no other transaction changes a locked row.

- Shared lock (S) — "read lock." Multiple transactions can hold S on the same row and read in parallel; no one can write while an S is held.
- Exclusive lock (X) — "write lock." Only one transaction; while held, no other transaction can read or write (can't get S or X).

Held \ Requested	Shared (S)	Exclusive (X)
Shared (S)	✅ granted	❌ denied
Exclusive (X)	❌ denied	❌ denied

Prerequisite 3 — Isolation Levels & the 3 read problems

The I in ACID controls how much concurrency is allowed. Three anomalies:

- Dirty Read — T-A reads a value T-B wrote but hasn't committed; if T-B rolls back, T-A read garbage.
- Non-Repeatable Read — T-A reads the same row twice and gets different values (another transaction committed an update in between).
- Phantom Read — T-A runs the same range query twice and gets a different set of rows (another transaction inserted a row in the range).

Isolation Level	Locking strategy	Dirty	Non-Repeatable	Phantom	Concurrency
Read Uncommitted	no locks	✗ possible	✗ possible	✗ possible	highest (read-only use)
Read Committed	S lock released immediately after read; X held till commit	✓ solved	✗ possible	✗ possible	high
Repeatable Read	S and X held till end of transaction	✓	✓ solved	✗ possible	medium
Serializable	Repeatable Read + range locks till end	✓	✓	✓ solved	lowest

Set per transaction (`SET TRANSACTION ISOLATION LEVEL ...`); otherwise the DB default applies (e.g., InnoDB → Repeatable Read).

Optimistic vs Pessimistic Concurrency Control

Optimistic Concurrency Control — uses Read Committed; resolves conflicts with versioning.

- Each row has a version (built-in in MySQL/InnoDB; add a column in Oracle and increment on update).
- Read the row + its version (no lasting lock). Do computation. To update: take an X lock, validate the version — if the DB version still equals the one you read, commit and bump the version; if it changed (someone else updated), roll back and retry.
- ✓ High concurrency, no deadlock. Best when contention is rare. ✗ Overhead of retry on conflict.

Pessimistic Concurrency Control — uses Repeatable Read or Serializable; holds locks for the whole transaction.

- Effectively serializes conflicting transactions (they wait for locks).
- ✓ Strong safety. ✗ Deadlocks possible; long-held locks can cause timeouts.

Deadlock illustration: T1 (read A, write B) and T2 (read B, write A). Pessimistic: T1 locks A, T2 locks B, each waits for the other → deadlock (both abort). Optimistic: locks are released right after each read, so both complete → no deadlock.

	Optimistic	Pessimistic
Isolation	≤ Repeatable Read (Read Committed)	Repeatable Read / Serializable
Mechanism	version validation	hold locks
Concurrency	higher	lower
Deadlock	✗ no	✓ possible
Best for	low contention	high contention

(The most popular pessimistic protocol is Two-Phase Locking — next section.)

0027 — TWO-PHASE LOCKING (2PL)

A form of pessimistic concurrency control, widely used. Every transaction runs in two phases:

- Phase 1 — Growing: transaction may only acquire locks (request from the lock manager; granted unless

already held). Lock count only grows.

- Phase 2 — Shrinking: transaction may only release locks; no new locks. Lock count only shrinks.

The graph is always "acquire → ... → release" — never interleaved acquire/release/acquire.

Three variants

Variant	Rule	Deadlock?	Cascading abort?	Concurrency
Basic 2PL	may release locks before commit (once shrinking starts)	✓ possible	✓ possible	highest
Conservative 2PL (static)	acquire all locks at the start; if any unavailable, wait and take none	✗ avoided	✓ possible	lowest (+ scheduler must know all locks upfront)
Strict / Rigorous 2PL	hold all locks (S and X) until end of transaction (commit/abort)	✓ possible	✗ avoided	medium

- Basic 2PL's two problems: deadlock (T1 holds A wants B, T2 holds B wants A; also possible on a single row via S→X upgrade contention) and cascading aborts (T1 updates A, releases the lock before committing, T2 reads A, then T1 aborts → T2 did a dirty read and must abort too).
- Strict/Rigorous 2PL is the most widely used in industry — cascading aborts are too expensive, and deadlocks are handled with a wait-for graph.

Deadlock detection/prevention strategies

1. Timeout — abort a transaction that waits "too long." Simple, but may falsely abort a valid transaction that's merely slow (no actual deadlock).
2. Wait-For Graph (WFG) — a directed graph with an edge $T_i \rightarrow T_j$ when T_i waits for T_j 's lock. Periodically check for a cycle = deadlock; remove edges as locks release. On a cycle, pick a victim to abort based on: effort already spent, effort left to finish, rollback cost (how many updates to undo), and how many cycles the transaction is in. *(Most widely used with 2PL.)*
3. Conservative 2PL — acquiring all locks upfront prevents deadlock entirely (but low concurrency).
4. Timestamp-based (older = higher priority):
 - Wait-Die: an older transaction waits for a younger one's lock; a younger one requesting an older's lock dies (aborts).
 - Wound-Wait: an older transaction wounds (aborts) the younger holder and takes the lock; a younger one waits for an older's lock.

0028 — OAUTH 2.0

OAuth = Open Authorization — an authorization framework enabling secure third-party access to a user's protected data (e.g., "Sign in with Google" on some website → you authorize that site to use your Google profile data).

4 actors

1. Resource Owner — the user (owns the data).

2. Client — the third-party app requesting access (e.g., Instagram).
3. Authorization Server — issues tokens (e.g., Google's auth server).
4. Resource (Hosting) Server — hosts the protected data (e.g., Google holding your profile).

Grant types (mechanisms to get a token)

Authorization Code Grant (most important & most used):

1. Registration: the client registers with the auth server → receives client ID + client secret (secret known only to client & auth server).
2. Authorize: user clicks "Sign in with Google"; client redirects to auth server calling `/authorize` with `response_type=code`, `client_id`, `redirect_uri` (optional — must match one registered), `scope` (space-separated data requested), and `state` (random anti-CSRF value).
3. User authenticates + gives consent → auth server redirects to `redirect_uri` with an authorization code + the same `state`.
4. Token: client calls `/token` with `grant_type=authorization_code`, the `code`, `redirect_uri`, `client_id`, `client_secret` → receives an access token (short-lived) + refresh token (long-lived).
5. Client uses the access token (in the `Authorization: Bearer` header) to fetch data from the resource server, which asks the auth server to validate it → returns data (or 401 if invalid).
6. Refresh: when the access token expires, call `/token` with `grant_type=refresh_token` + refresh token → new access token (no username/password needed).

Why `state` (CSRF protection): Without it, an attacker can obtain *their own* authorization code, then trick the client into using it, so the victim ends up signed in as the attacker (uploads go to the attacker's Drive, etc.). The client generates a hard-to-guess `state`, sends it in `/authorize`, and rejects any callback whose returned `state` doesn't match — defeating the forged response.

Access vs Refresh token: access token is short-lived (e.g., 15–60 min) and grants data access; refresh token is long-lived and only mints new access tokens.

Other grant types:

- Implicit Grant — `/authorize` with `response_type=token` returns the access token directly (one step, no refresh token). Not recommended.
- Resource Owner Password Credentials — client sends the user's username/password directly to `/token` (`grant_type=password`). No `/authorize` step. Only for highly-trusted first-party clients.
- Client Credentials — when the client is itself the resource owner (machine-to-machine); `/token` with `grant_type=client_credentials` + client ID/secret. No refresh token needed.

0029 — CRYPTOGRAPHY

Foundational for HTTPS, end-to-end-encrypted chat, and JWT. Encryption turns readable plaintext into unreadable ciphertext using a key + algorithm; decryption reverses it with a key.

Symmetric vs Asymmetric

	Symmetric	Asymmetric
Keys	Same key for encrypt & decrypt	Two keys: public + private
Algorithms	DES (56-bit, cracked ~2005), AES (128/192/256-bit)	RSA (2048-bit), DSA, Diffie-Hellman, ECDHE
Speed	Fast, low compute	Slow, compute-intensive (large keys)
Best for	Bulk data (chat, HTTPS payloads)	Key exchange, digital signatures
Weakness	Key distribution — how to share the key securely? Server must manage one unique key per client	Slow — unsuitable for bulk data

Both are used together (like left & right hand): chat uses symmetric AES for bulk encryption, but the AES key is exchanged securely using asymmetric Diffie-Hellman.

AES internals (high level)

A block cipher processing data in 128-bit blocks. Terminology: state array (a 128-bit block → 4×4 byte matrix), word (4 bytes), round key (4 words).

- Key expansion: the 128-bit key → 44 words (each new word = XOR of earlier words, some passed through a function).
- Rounds: initial AddRoundKey, then N rounds of SubBytes → ShiftRows → MixColumns → AddRoundKey. Rounds by key size: 128-bit → 10, 192 → 12, 256 → 14 (more bits = more security, more compute). Decryption runs these steps in reverse. ("Adding salt" ≈ adding more rounds/keys.)

Diffie-Hellman key exchange

Lets two parties agree on a shared secret over an insecure network (uses asymmetric math):

1. Both publicly agree on a prime p and a primitive root g (an attacker can see these).
 2. Each picks a secret private key (a , b) — never shared.
 3. Each computes a public key: $A = g^a \bmod p$, $B = g^b \bmod p$ — and exchanges them.
 4. Each computes the shared secret: $A^b \bmod p = B^a \bmod p$ → same value.
- The attacker sees p, g, A, B but not a or b ; recovering them (discrete log) is infeasible for large private keys → security rests on choosing big private keys.

Digital Signatures (authentication + integrity)

1. Sender: hash the plaintext ($hash$), then sign the hash with the sender's private key → signature. Send $\{plaintext, signature\}$.
 2. Receiver: recompute the hash of the plaintext; verify the signature with the sender's public key → yields the original hash.
 - If the two hashes match → data is unmodified (integrity) and truly from the sender (authentication); only the sender's private key could have produced a signature verifiable by their public key.
- Signing = private key, verifying = public key (asymmetric). Used by JWT (JWS).

0030 — JWT (JSON WEB TOKEN)

A compact, stateless, self-contained way to transmit information between parties as a signed JSON object. Used for authentication, authorization, and SSO (Single Sign-On).

JWT vs Session ID

	Session ID (JSESSIONID)	JWT
State	Stateful — session stored in DB	Stateless — all info in the token
Per-request cost	DB/cache lookup to validate + fetch roles/expiry	No DB — validate signature locally
Distributed systems	Requires session sync across DB clusters	No shared state needed
Latency	Extra DB query per request	Faster

Structure: `header.payload.signature` (three base64url parts joined by `.`)

- Header: metadata — `typ` (always `JWT`), `alg` (signing algorithm, e.g., RSA (asymmetric) or HMAC (symmetric)).
- Payload: claims (data). Three kinds:
 - Registered (reserved, standard meaning): `iss` (issuer), `sub` (subject/user), `aud` (audience/recipient), `exp` (expiry), `nbf` (not-before), `iat` (issued-at), `jti` (unique token ID).
 - Public (custom, understood by multiple parties): e.g., `email`, `country`.
 - Private (custom, understood only by the issuer/auth server internally).
 - ⚠ Never put confidential data (passwords) in the payload — it's only encoded, not encrypted.
- Signature: `sign( base64url(header) + "." + base64url(payload), key )`. With HMAC the same secret signs & verifies; with RSA, sign with private key, verify with public key. Then base64url-encode and append.

JWT vs JWS vs JWE: In practice "JWT" means JWS (JSON Web Signature — signed). JWE = JSON Web Encryption (payload encrypted, not just encoded). A JWT with `alg: none` (no signature) is an Unsecured JWT and must be rejected.

Transport: always sent client→server in the `Authorization` header as `Bearer <token>` (vs `Basic <base64 user:pass>`).

Advantages

Compact (fits in a header), stateless/self-contained, digitally signable (integrity), built-in expiry (`exp`), custom claims (roles). Enables SSO: authenticate once → reuse the same JWT across app1/app2/app3 (each verifies signature + reads user info from the payload).

Challenges (interview-critical)

- Token invalidation — since JWT is stateless, you can't easily revoke a token before `exp` (e.g., to block a fraudulent user). Options:
 - Blacklist revoked token IDs (`jti`) in DB/cache — but reintroduces the DB/cache lookup JWT was meant to avoid.

- Rotate the signing secret — invalidates the bad token, but logs out all genuine users too.
 - Short-lived tokens + one-time use (most popular) — e.g., valid 5–10 min, used once, then re-issued.
2. Encoded, not encrypted — the payload is base64-decodable → less secure. Use JWE to encrypt the payload.
 3. Unsecured JWT (`alg: none`) — must always be rejected.
 4. JWK exploit — the header may carry a JWK (embedded public key). Never verify using the public key from the token's own JWK — an attacker could tamper the payload, re-sign with *their* private key, and attach *their* public key. Instead, use the header's `kid` (Key ID) to look up the correct public key from the auth server's well-known JWKS (whitelisted keys). ⇒ Choose a reputable third-party auth provider.
-

0031 — TICKET-BOOKING CONCURRENCY FAILURE (CASE STUDY)

A 10-minute analysis of why a ticket-booking app might crash when a concert sale opens at 12pm.

Baseline: Load balancer → multiple ticket-service instances, each protected by a thread-pool executor + queue sized to its capacity.

The failure chain

1. Thundering Herd — at 12pm a massive traffic spike hits at once. The LB distributes it, but every instance's threads and queues fill up → new requests rejected.
2. Retry storm — rejected clients retry, so load = new requests + retries, compounding the pileup.
3. Cascading latency/timeouts — running at full capacity increases latency (10s → 20s per request); in-flight and queued requests time out; those clients also retry → more load.
4. Auto-scaling isn't enough — new instances spin up but the retry flood immediately saturates them too.

Solutions

1. Exponential Backoff for retries — wait `base × 2^n` (n = failure count), e.g., 200ms, 400ms, 800ms... ❌
Alone, all clients using the same formula still retry simultaneously.
2. Backoff + Jitter — add randomness: `wait = min(max_wait, random(0, base × 2^n))`. Spreads retries out so they don't align.
3. Rate Limiter (Token Bucket) at the API Gateway — the first line of defense. Only as many requests as there are tokens (sized to real capacity) pass through; the rest wait/reject. This keeps existing requests below the latency-cliff and spreads an instant burst across an interval (e.g., a minute).

Combined fix: Rate limiter (token bucket) at the gateway + exponential backoff + jitter on retries + auto-scaling.

0032 — API GATEWAY

Definition: A single entry point that accepts client API requests and routes them to the correct backend service based on the API endpoint (`/api/invoice` → invoice service, `/api/order` → order service).

API Gateway vs Load Balancer: A load balancer just distributes traffic across multiple instances of the same service — it can't understand an API path and decide *which* service to route to. An API gateway is API-aware and intelligent; it sits before the load balancers.

Key features

- API Composition — aggregate responses from multiple microservices into one (e.g., "My Orders" page: mobile fetches product + invoice; desktop also fetches ratings + reviews + recommendations). The gateway calls the needed services and returns a single response, removing complexity from the client (heavily used by Netflix).
- Authentication — validate the client's token (e.g., OAuth 2.0) with the auth server *once* at the edge, instead of duplicating auth in every service.
- Rate Limiting — burst limit (max concurrent requests before 429), throttling (granular per-API/per-user limits, e.g., `/api/invoice` ≤ 10/min per user), IP blocking, and API queues (hold overflow requests to survive a thundering herd).
- Service Discovery — track service locations (IP/port change as they scale up/down). Two approaches: services register/deregister themselves, or the discovery service health-checks and keeps only live instances (software: Eureka, Zuul). The gateway asks discovery for a service's location before routing.
- Also: request/response transformation, response caching, logging.

"Single entry point" yet handles millions/sec — how?

Uses Regions and Availability Zones (AZs):

- A Region (e.g., Mumbai) contains multiple AZs (isolated areas, each with its own data center, sharing no resources).
- Each AZ runs the full stack (services + their load balancers). If one AZ fails, its traffic shifts to another AZ; if all AZs fail, the Region is down → traffic shifts to another Region. ⇒ no single point of failure.
- The API gateway itself is replicated per region. A DNS-based load balancer (AWS Route 53, Azure Traffic Manager) distributes traffic across regional gateways based on latency/compliance rules. (DNS itself isn't a SPOF — it's a distributed hierarchy; see 0034.)

0033 — SERVICE MESH

Problem: How do two microservices (A → B) communicate? Brute-force, service A needs 7 capabilities built in:

1. Service discovery (get B's location),
2. Client-side load balancing (pick an instance),
3. Authentication/Authorization (is A allowed to call B?),
4. Circuit breaker (stop calling a failing B),
5. Retry (retry transient 5xx errors, not 4xx),
6. Deployment strategy support (canary/blue-green traffic splitting),
7. Telemetry (record latency, error rate, traffic; logs/samples).

Building all 7 into every service is a huge burden → Service Mesh solves it.

Architecture (Istio example on Kubernetes)

- Data Plane — Sidecar Proxy: each pod runs the microservice instance + a sidecar proxy (Istio: Envoy) that intercepts all inbound/outbound traffic (no code change, no network hop to the sidecar — it's interception). Sidecars talk directly to each other and carry all 7 capabilities. mTLS encrypts sidecar-to-sidecar traffic.
 - Control Plane: configures the sidecars —
 - Config Manager (Istio: Galley) — validates user config (YAML/UI: enable circuit breaker, retry count, etc.).
 - Traffic Controller (Istio: Pilot) — pushes config to the sidecars.
 - Security Manager (Istio: Citadel) — issues TLS certificates/keys for authn/authz between sidecars.
 - Telemetry — pulls metrics from sidecars → observability dashboards.
 - Key point: control plane ↔ data plane sync happens only on config change, not per request.
 - To call B, service A just says "talk to service B" (by name, no IP/port); the sidecar uses its service-discovery config + load balancer to pick an instance and forward.
-

0034 — DNS (DOMAIN NAME SYSTEM)

IP address = unique numeric label per device (IPv4/IPv6). Domain name = human-readable address (google.com). DNS = the translator from domain name → IP.

Name hierarchy (read bottom-up)

`www.conceptandcoding.com.` → `.` root · `.com` Top-Level Domain (TLD) · `conceptandcoding` Second-Level Domain (SLD) · `www` subdomain. The whole thing is the FQDN (Fully Qualified Domain Name); `conceptandcoding.com` is the domain name.

DNS records

- Record name — the domain/subdomain.
- CNAME (Canonical Name) — an alias mapping one domain to another (mostly at subdomain level; e.g., `www.google.com` → `google.com`). No IP directly — triggers another lookup.
- A record (Address) — maps a name to an actual IP address.

Resolution — Recursive

1. Stub resolver (OS DNS client) checks its local cache (`ipconfig /displaydns`).
2. Miss → query the DNS Resolver (ISP's, or Google's `8.8.8.8`).
3. Resolver (cache miss) → asks a Root server (only 13 worldwide, A–M, each run by a different org, e.g., VeriSign). Root returns the TLD server's IP (e.g., `.com`).
4. Resolver → TLD server (`.com`, run by VeriSign). Returns the Authoritative Name Server (NS record) for the domain.

5. Resolver → Authoritative server (e.g., GoDaddy, the registrar — which registered the NS records with the TLD registry). Returns the A record / IP.
 6. Resolver caches and returns the IP to the client.
- "Recursive" because the client makes one request and the resolver does all the walking.

DNS Zones

An authoritative server would otherwise hold records for *all* subdomains (unbounded: mail., blog., admin., a.b.c....), and a hot subdomain overloads it. Zones split authority: offload a busy subdomain (e.g., `mail.`) to its own authoritative server, so the main server just forwards those requests.

Recursive vs Iterative

- Recursive: the resolver contacts root → TLD → authoritative on the client's behalf.
 - Iterative: the DNS client does the walking — the resolver just returns the next server's IP (root → then client asks TLD → then authoritative) each step.
-

0035 — HOW MANY MICROSERVICES? (SIZING A DECOMPOSITION)

There is no fixed number. Answer by stating what a good microservice must achieve, then apply a method.

Expectations from each microservice: (1) loosely coupled (change one without changing another), (2) independently code/test/deploy (own team, evolves independently), (3) less communication overhead (not chatty — chattiness → latency + coupling), (4) scale independently (scale one without scaling another).

Method — Domain-Driven Design (DDD)

1. Understand the domain (e.g., a chat application) with domain experts — clarify the problem.
2. Find subdomains via Event Storming: the whole team (experts + devs + testers) whiteboards —
 - List all events (User Registered, User Login, Message Sent, Message Delivered, Message Deleted...).
 - Sequence them & find missing events (add User Logout, Message Received...).
 - Group into Bounded Contexts.
3. Bounded Context — the *same object* can mean different things in different contexts, so they belong to different boundaries. (Analogy: a sandwich in a restaurant has value/you'll pay & eat it; the same sandwich in the garbage has zero value — different boundaries.) E.g., `User` in *User Management* (auth/permissions) is a richer object than `User` in *Notification* (just a user ID + status) → separate contexts.
4. One microservice per bounded context → e.g., User Management, Message, Notification. Minimal duplication (a user ID appearing in two services) is acceptable; heavy dependency is not.

Avoid the Distributed Monolith: if the four expectations aren't met, splitting just adds network overhead. Amazon Prime Video famously split audio & video into two tightly-coupled services, hit huge overhead, and merged them back — gaining ~90% efficiency (still microservices overall, just right-sized).

0036 — WEB ATTACKS: CSRF, XSS, CORS, SQL INJECTION

CSRF — Cross-Site Request Forgery

Tricks an already-authenticated user's browser into making an unwanted request to a site where they're logged in. The browser automatically attaches the session cookie, so the malicious request looks legitimate (e.g., a hidden `/transfer` call). Applies mainly to session/stateful auth.

- Protection: CSRF token — the server issues a token known only to the legitimate page; genuine forms append it, attackers can't guess it, so the server rejects forged requests.

XSS — Cross-Site Scripting

An attacker injects a malicious script into a page viewed by other users (e.g., posting `<script>...</script>` as a comment). When others load the page, their browser executes the script — commonly to steal cookies/sessions (`<script>...document.cookie...</script>` sends the victim's session to the attacker) or deface the site.

- Protection: escape user input (convert `<`, `>` etc. so the browser treats it as text, not code) and validate allowed input.

CORS — Cross-Origin Resource Sharing

Not an attack — a security feature. Restricts web pages from calling a different origin unless the server allows it. Origin = protocol + domain + port (any difference = different origin, e.g., `http` vs `https`, `:8080` vs `:9090`, `sub.localhost` vs `localhost`).

- The server whitelists allowed origins via the `Access-Control-Allow-Origin` header (+ allowed methods/headers). Acts as a first line of defense — a malicious page on a different, non-whitelisted origin is blocked.

SQL Injection

An attacker manipulates a SQL query by injecting input into a user field. If input is concatenated directly (`SELECT * FROM users WHERE name = '<input>'`), passing `' OR 1=1` makes the `WHERE` always true → dumps all rows; worse inputs can read table/column names, access unauthorized data, or DROP tables.

- Protection: parameterized queries / prepared statements — bind the input as a value, never as executable SQL (e.g., `setParameter`), so `' OR 1=1` matches nothing.
-